

Documentation for Data project: Influence of Temperature deviations on energy consumption deviations in France in 2017

Tobias Habacker

October 7, 2025

1 Introduction

The goal of this short project is to analyze the influence of the temperature deviation from the mean temperature on the energy consumption deviation in France in the year 2017. The main question we seek to answer is:

**Can we say with a certain probability that we will have a surplus
or a deficit in the grid depending on the temperature deviation?**

Our hypothesis is that there is some influence, although it is not clear in the first place, how strong/significant it is - we aim to quantify this relation. There may be more possible predictors for the consumption deviation than only temperature.

2 Tools, modules and methods

Mainly Python3 is used with the following modules/packages:

- `numpy` : standard math library
- `pandas` : important package for working with data. Provides series and data frames, which is the most important data structure used in the program.
- `matplotlib` : used for plotting the linear regression.
- `statsmodels` : used the linear regression. Provides statistical tools and key figures related to the quality of the linear fit.

3 Files

Temperature data between 1974 and 2017 from the following weather stations:

- LFPG.csv - Paris (Charles de Gaulle airport)
- LFPO.csv - Paris (Paris-Orly airport)
- LFMT.csv - Montpellier
- LFRN.csv - Rennes
- LFBD.csv - Bordeaux
- LFBO.csv - Toulouse
- LFOJ.csv - Orleans
- LFRM.csv - Le Mans
- LFSD.csv - Dijon
- LFOH.csv - Le Havre
- LFBZ.csv - Biarritz-bayonne
- LFMP.csv - Perpignan
- LFML.csv - Marseille

- LFMD.csv - Cannes
- LFST.csv - Strasbourg
- LFLY.csv - Lyon
- LFSO.csv - Nancy
- LFQQ.csv - Lille
- LFMN.csv - Nizza
- LFRS.csv - Nantes

Weather data is taken from

https://mesonet.agron.iastate.edu/request/download.phtml?network=FR_ASOS.

Data for the average energy consumption deviation in France (2017):

- Consumption deviation 2017.csv

Python script for data analysis:

- DataAnalysis.py

4 Documentation of the program

4.1 Functions

Function	Description
<code>load_weather_data()</code>	Reads weather data, cleans it and adds time components.
<code>choose_stations()</code>	Asks the user if he wants to use a single weather station. If the user declines, the average over all stations will be taken.
<code>compute_mean()</code>	Computes averages over given columns using the <code>groupby()</code> -method from the <code>pandas</code> -package.
<code>load_consumption_data()</code>	Reads consumption data and corrects the known error if necessary.
<code>corr()</code>	Computes the Pearson correlation of two columns.
<code>scatterplot()</code>	Creates scatter plots for summer/winter tables.
<code>lin_reg_statsmodels()</code>	Performs linear regression using <code>statsmodels</code> , outputs a summary of the regression with all the relevant data. Plots the result and confidence bands for the regression line.

4.2 The main()-method

1. Loading weather data: the function `choose_stations` is applied to let the user choose the weather station. The function `load_weather_data` loads the needed csv-files from the current folder and saves it as a dataframe. The entries in the "valid" column, which contains the date and time are converted to `datetime`-objects, to get access to the year, month, day, hour and minute of the date. These keys are then added as separate columns.
2. Separate data from 2017 and before: separate tables are created, one with data from all years before 2017, another with only data from 2017.
3. Compute averages: at first, daily averages are computed using the `compute_mean`-function, then also average temperatures for a certain day in all years before 2017. Here, the `groupby()` method from the `pandas` package is applied.
4. Loading consumption data:: data is loaded from the file "Consumption deviation 2017.csv" with the function `load_consumption_data`. Since there is a **mistake** in this file - the entry for the 4th of June 2017, 00:00 - the user is asked if the incorrect value should be deleted. Daily averages for the consumption are computed.

5. Merge tables: the table for energy consumption is extended with the average daily temperatures and daily temperature deviation.
6. Winter/Summer: separate tables for summer and winter are created.
7. Correlation: the correlation of "Temperature deviation (2017)" and "Avg. daily cons. deviation (2017)" is computed using the `corr()` -method from `pandas`.
8. Scatterplot, linear regression and regression plot: a scatterplot for the data is created. Then the function `linear_regression_plot` applies a linear regression for the columns "Temperature deviation (2017)" and "Avg. daily cons. deviation (2017)". The linear regression is provided by the `statsmodels` -package, which also provides all the relevant data as R^2 - and p -values and confidence bands for the estimates. The function `linear_regression_plot` also plots the data and regression line using the module `matplotlib`.

5 Statistical analysis

5.1 Statistical tools

We use linear regression for the statistical analysis. For this we assume that there is an approximate linear relation between the temperature deviation (X) and the energy consumption deviation (Y) in France in 2017. We then have

$$Y \approx \beta_0 + \beta_1 X,$$

where β_0 (intercept) and β_1 (slope) are the parameters we have to fit using our data. Important statistical measures are:

- R^2 : the R^2 -value quantifies how well the regression model's predictions fit the actual data points. More detailed, it shows the percentage of the variance in Y that is accounted for by X . It is the square of the **Pearson correlation**.
- p : the p -value for β_i with $i \in \{0, 1\}$ is the probability that the null hypothesis $H_0 : \beta_i = 0$ is true, i.e. $1 - p$ indicates the probability that our null hypothesis can be rejected and there is some linear relation between X and Y . Usually, a model with $p < 0.05$ is called **significant**.
- 95% confidence intervall (ci): an intervall I around an estimated parameter $\beta \in \mathbb{R}$, such that that the real value for the parameter is in I with probability of 95%. If the error is assumed to be normally distributed, we have

$$I = [\beta - q \cdot SE(\beta), \beta + q \cdot SE(\beta)],$$

where $q \approx 2$ and $SE(\beta)$ is the standard error/standard deviation of β . Confidence intervalls can also be computed for estimated values $\hat{y} = \beta_0 + \beta_1 x_0$ by our model. Plotting all 95%-CIs around estimations gives a "confidence band" around the regression line.

5.2 Example

Summer: Correlation of Temperature deviation (2017) and Avg. daily cons. deviation (2017) : -0.3966655702258452

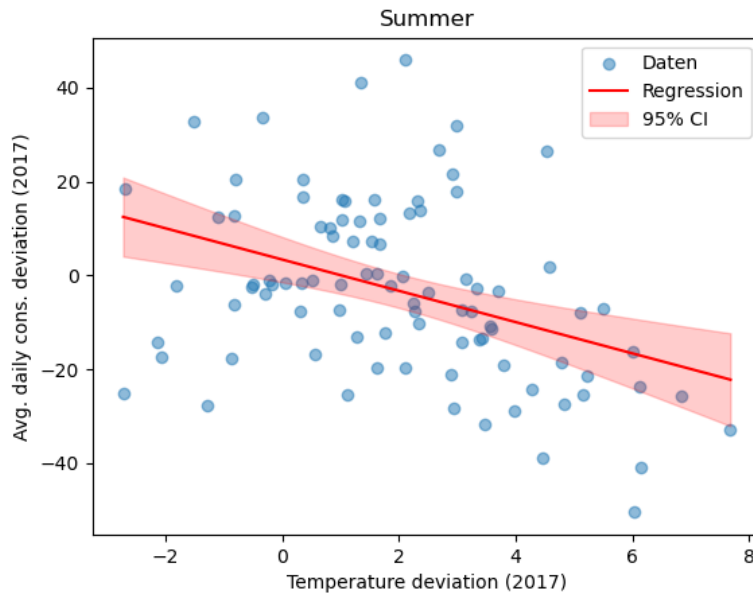
===== SUMMER =====

OLS Regression Results

```
=====
Dep. Variable:      Avg. daily cons. deviation (2017)    R-squared: 0.157
Model:              OLS                                Adj. R-squared: 0.148
Method:             Least Squares                      F-statistic: 16.81
Date:               Tue, 07 Oct 2025                   Prob (F-statistic): 9.06e-05
Time:               10:13:14                           Log-Likelihood: -392.57
No. Observations:   92                                 AIC: 789.1
Df Residuals:       90                                 BIC: 794.2
Df Model:           1
Covariance Type:    nonrobust
=====
```

	coef	std err	t	P> t	[0.025	0.975]
intercept	3.3884	2.426	1.397	0.166	-1.431	8.208
slope	-3.3299	0.812	-4.099	0.000	-4.944	-1.716

```
=====
Omnibus:                2.825    Durbin-Watson:                1.410
Prob(Omnibus):           0.244    Jarque-Bera (JB):         2.258
Skew:                    0.369    Prob(JB):                 0.323
Kurtosis:                3.212    Cond. No.                  4.19
=====
```



This is a sample output of the programme for the weather station in Marseille in summer. The incorrect entry in the energy consumption table has been deleted. The output shows:

- Pearson-correlation: -0.3966655702258452
- $R^2 = 0.157$
- p -value for intercept (β_0) : 0.166, for slope (β_1) : 0.000 (rounded)
- 95% CI: for intercept $I = [-1.431, 8.208]$, for slope $I = [-4.944, -1.716]$
- The confidence band is plotted in red around the regression line.

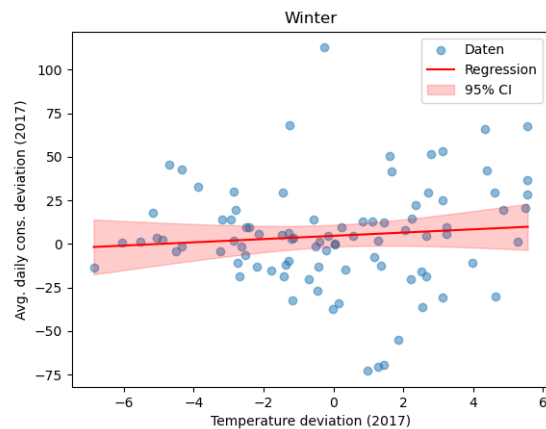
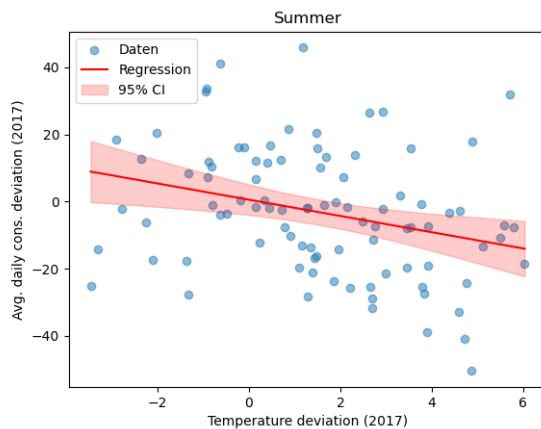
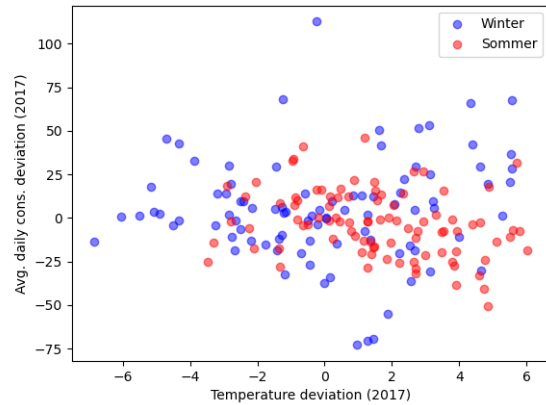
6 Results and Interpretation

In the following analysis, we consider weather stations in the seven largest cities in France (Paris-CDG, Marseille, Lyon, Toulouse, Nizza, Nantes, Montpellier) - this can be achieved by commenting out all other stations in the program code. The output of our program is:

```
===== SUMMER =====
                        OLS Regression Results
-----
Dep. Variable:   Avg. daily cons. deviation (2017)   R-squared:      0.006
Model:           OLS                               Adj. R-squared:  0.076
Method:           Least Squares                     F-statistic:    8.495
Date:            Tue, 07 Oct 2025                   Prob (F-statistic): 0.00449
Time:            11:43:01                           Log-Likelihood: -396.29
No. Observations: 92                               AIC:          796.6
Df Residuals:    90                               BIC:          801.6
Df Model:         1
Covariance Type: nonrobust

===== WINTER =====
                        OLS Regression Results
-----
Dep. Variable:   Avg. daily cons. deviation (2017)   R-squared:      0.009
Model:           OLS                               Adj. R-squared: -0.002
Method:           Least Squares                     F-statistic:    0.7828
Date:            Tue, 07 Oct 2025                   Prob (F-statistic): 0.379
Time:            11:43:03                           Log-Likelihood: -433.29
No. Observations: 99                               AIC:          878.6
Df Residuals:    88                               BIC:          875.6
Df Model:         1
Covariance Type: nonrobust

=====
coef    std err      t    P>|t|    [0.025    0.975]
-----
intercept    0.5762    2.293    0.251    0.802    -3.979    5.132
slope       -2.4197    0.830   -2.915    0.004   -4.069   -0.778
```



6.1 Correlation

For the correlation, we obtain the following results:

season	correlation	interpretation
winter	+0.094	very weak positive correlation
summer	-0.294	weak negative correlation

Although the correlation is weak, the signs are correct:

- In winter there is a positive correlation: High temperatures (positive temperature deviations) in winter should lead to less energy consumed for heating → **Surplus** in the grid (positive entry in consumption deviation)
- In summer there is a (stronger) negative correlation: High temperatures (positive temperature deviations) in summer should lead to more energy consumed for cooling → **Deficit** in the grid (negative entry in consumption deviation)

6.2 Linear regression

With the values for slope and intercept from the output of the program, we obtain the linear models

$$\widehat{Y}_s = 0.58 - 2.42X_s$$

$$\widehat{Y}_w = 4.65 + 0.93X_w$$

as predictions Y_w for winter and Y_s for summer. For winter, we have $R^2 = 0.009$ and p -values of 0.147 for the intercept and 0.379 for the slope. Assuming a 5%-significance level, this means that that we cannot prove a significant relationship for winter.

For summer, we have the following:

key figure	value	meaning
slope	-2.42 ± 0.83	For every 1 °C temperature deviation, avg. consumption deviation decreases by 2.4 MW on average.
p -value	$0.004 (< 0.05)$	→ statistically significant
R^2 -value	0.086	Approximately 9% of the variance in consumption is explained by temperature.
95%-CI (slope)	$I_s = [-4.069, -0.770]$	Assuming there is a linear relation, the true value for the slope is in I_s with a probability of 95%.

This shows that that there is a statistically significant correlation of temperature deviation with consumption deviation in summer - although the influence is not very strong. In the plots above one can also see the confidence bands around the regression line. Assuming the relation between our data is linear and given some temperature deviation x_0 , the real value lies in the confidence band below or above the estimate $\widehat{Y}_0 = 0.58 - 2.42X_0$ with a probability of 95%.

6.3 Remark

If we do not delete the mistake in the consumption data, we obtain the following output for the correlation:

```
Winter: Correlation of Temperature deviation (2017) and Avg. daily cons. deviation (2017) : 0.09384975844765263
Summer: Correlation of Temperature deviation (2017) and Avg. daily cons. deviation (2017) : 0.06333330668388702
```

We can see that the correlation for both summer and winter is postive in this case. This is not realistic, as it should be negative, since higher temperatures in summer lead to more cooling, therefore higher energy consumption, which leads to a deficit in the grid.

7 Conclusion

The analysis shows a statistically significant negative relationship between temperature deviation and the deviation of the energy balance in the power grid during the summer months. The negative slope of $-2.4 \text{ MW} / ^\circ\text{C}$ implies that higher-than-average temperatures lead to a larger energy deficit. This is consistent with the physical expectation that higher temperatures increase electricity demand due to cooling and air conditioning.

In contrast, the winter data show a slightly positive but statistically insignificant relationship. Warmer winter days tend to produce a small energy surplus (lower heating demand), yet the effect is weak ($R^2 \approx 0.01$) and not significant. This suggests that the temperature effect on the grid balance is minor compared to other influencing factors such as industrial activity, calendar effects, or fluctuations in renewable energy production.

A reason for the difference between summer and winter could be that cooling in summer mostly depends on electricity, since air conditioning, ventilation, and refrigeration systems all depend on electric power. Heating in winter relies not primarily on electricity. There are other resources and methods, such as natural gas or oil, wood or other local fuels in rural areas and district heating.

Therefore the effect may not be as strong in winter. Overall, temperature deviation explains only a small fraction of the daily variation in the power grid's energy balance. Still, the results confirm that temperature fluctuations can have a measurable impact on grid stability, particularly during the summer months when cooling demand is sensitive to temperature changes.

Future improvements of the analysis could include:

- Adding further meteorological predictors (e.g., humidity, solar radiation)
- Considering calendar and behavioral factors (weekdays, holidays)
- Extending the study to multiple years to identify longer-term patterns
- Applying nonlinear or multiple regression models to capture more complex dependencies

Such extensions would provide a more comprehensive understanding of how temperature variations influence grid load and could help to refine energy demand forecasting under changing climate conditions.

8 Appendix

8.1 Program code

```

1 import pandas as pd
2 import matplotlib.pyplot as plt
3 import numpy as np
4 import statsmodels.api as sm
5
6 #load data: loads weather data and returns it as a dataframe: either for a single station or
7 #taking the average over multiple stations
8 def load_weather_data(stations: dict=None, single_station=None):
9     if single_station:
10         df = pd.read_csv(stations[single_station], low_memory=False)
11         #missing data is encoded with an 'M', so .to_numeric() is needed here
12         df["tmpc"] = pd.to_numeric(df["tmpc"], errors="coerce")
13         df.drop(columns=["station"], inplace=True)
14     else:
15         # read .csv files with weather data in stations
16         dfs=[]
17         for f in stations.values():
18             df = pd.read_csv(f, low_memory=False)
19             df["tmpc"] = pd.to_numeric(df["tmpc"], errors="coerce")
20             dfs.append(df)
21         #only keep columns 'valid' & 'tmpc'
22         dfs_tmp = [df[["valid", "tmpc"]] for df in dfs]
23         merged_df = pd.concat(dfs_tmp)
24         # calculate the mean over all stations
25         df = merged_df.groupby('valid', as_index=False)['tmpc'].mean()
26
27 #use datetime to access year, month, day and minute individually
28 df["valid"] = pd.to_datetime(df["valid"])
29
30 #add grouping keys year month, day, hour, minute
31 df["year"] = df["valid"].dt.year
32 df["month"] = df["valid"].dt.month
33 df["day"] = df["valid"].dt.day
34 df["hour"] = df["valid"].dt.hour
35 df["minute"] = df["valid"].dt.minute
36 df.drop(columns=["valid"], inplace=True)
37 return df
38
39 def choose_stations(stations):
40     choice=""
41     while choice not in ["y","n"]:
42         choice = input("Would you like to pick a single station by yourself? Else we take the
43             average over all available stations. y/n ")
44
45     if choice == "y":
46         station=""
47         while station not in stations.keys():
48             station = input(f"Which weather station should be used? You can choose between {'', ' '.
49                 join(stations.keys())}: ")
50         return station
51     else:
52         return None
53
54 def compute_mean(df, grouping_keys: list, col: str, name: str):
55     """computes the mean over entries of a given data frame

```

```

53         df:                the data frame used
54         grouping_keys:    a list containing the keys for grouping the data frame
55         col:              the column of which the mean should be calculated
56         name:            new name for the column with the means""
57         return df.groupby(grouping_keys)[col].mean().reset_index().rename(columns={col:name})
58
59
60 #load energy consumption
61 def load_consumption_data(file: str):
62     cons_dev = pd.read_csv(file, sep=";")
63     cons_dev["MW"] = pd.to_numeric(cons_dev["MW"], errors="coerce")
64
65     #mistake in the given data
66     decision=""
67     while decision not in ["y","n"]:
68         print("\nTable of Energy Consumption deviation (2017), 4th of June 2017, 00:00:")
69         print(cons_dev.loc[(cons_dev["timestamp (UTC)"] == "04.06.2017 00:00")])
70         decision = input("\nThe entry for the deviation in consumption on 4 June 2017 at 00:00 is
71             incorrect. Delete incorrect value? y/n ")
72
73     if decision == "y":
74         cons_dev.loc[7386, 'MW'] = float("nan")
75
76     #use datetime to access year, month, day and minute individually
77     cons_dev["timestamp (UTC)"] = pd.to_datetime(cons_dev["timestamp (UTC)"], dayfirst=True)
78
79     #add grouping keys month, day, hour, minute, drop the original timestamp
80     cons_dev["month"] = cons_dev["timestamp (UTC)"].dt.month
81     cons_dev["day"] = cons_dev["timestamp (UTC)"].dt.day
82     cons_dev["hour"] = cons_dev["timestamp (UTC)"].dt.hour
83     cons_dev["minute"] = cons_dev["timestamp (UTC)"].dt.minute
84     cons_dev.drop(columns=["timestamp (UTC)"], inplace=True)
85     return cons_dev
86
87 def corr(df, k1: str, k2: str):
88     corr = df[k1].corr(df[k2])
89     print("Correlation of " + k1 + " and " + k2 + " : " , corr)
90     return corr
91
92 #scatterplot, s1 and s2 are the relevant columns
93 def scatterplot(df1, df2, s1,s2):
94     plt.scatter(df1[s1], df1[s2], c="blue", alpha=0.5, label="Winter")
95     plt.scatter(df2[s1], df2[s2], c="red", alpha=0.5, label="Sommer")
96     plt.legend()
97     plt.xlabel(s1)
98     plt.ylabel(s2)
99     plt.autoscale()
100     plt.show()
101
102 # linear regression and plot using statsmodels
103 def lin_reg_statsmodels(df, x_col, y_col, title):
104     df = df.dropna(subset=[x_col, y_col])
105     x = df[x_col]
106     y = df[y_col]
107
108     X = sm.add_constant(x)
109     X.columns = ["intercept", "slope"]
110     model = sm.OLS(y, X).fit()
111     print(f"\n===== {title.upper()} =====")
112     print(model.summary())
113
114     x_fit = np.linspace(x.min(), x.max(), 100)
115     X_fit = sm.add_constant(x_fit)
116     y_fit = model.predict(X_fit)
117     ci = model.get_prediction(X_fit).conf_int()
118
119     plt.scatter(x, y, alpha=0.5, label="Daten")
120     plt.plot(x_fit, y_fit, 'r-', label="Regression")
121     plt.fill_between(x_fit, ci[:, 0], ci[:, 1], color='red', alpha=0.2, label="95% CI")
122     plt.xlabel(x_col)
123     plt.ylabel(y_col)
124     plt.title(title)
125     plt.legend()
126     plt.show()
127
128 def main():
129     #weather stations available for the analysis
130     weather_stations = {"Paris-CDG": "LFPG.csv",
131         "Paris-Orly": "LFPO.csv",
132         "Marseille": "LFML.csv",
133         "Lyon": "LFLY.csv",
134         "Toulouse": "LFB0.csv",
135         "Nizza": "LFMN.csv",
136         "Nantes": "LFRS.csv",
137         "Strasbourg": "LFST.csv",
138         "Montpellier": "LFMT.csv",
139         "Rennes": "LFRN.csv",

```



```

139         "Bordeaux": "LFBD.csv",
140         "Orleans": "LF0J.csv",
141         "Le Mans": "LFRM.csv",
142         "Dijon": "LFSD.csv",
143         "Le Havre": "LFOH.csv",
144         "Biarritz-bayonne": "LFBZ.csv",
145         "Perpignan": "LFMP.csv",
146         "Cannes": "LFMD.csv",
147         "Nancy": "LFSO.csv",
148         "Lille": "LFQQ.csv",
149     }
150
151     # 1. load weather data
152     station = choose_stations(weather_stations)
153     df = load_weather_data(stations=weather_stations, single_station=station)
154
155     #2. exclude the year 2017, separate table for 2017, drop columns from the table for 2017
156     df_no2017 = df[df["year"] < 2017].copy()
157     df_2017 = df[df["year"] == 2017].copy()
158
159     #3. Compute means
160
161     #3.1 Daily mean temperatures
162     df_daily = compute_mean(df_no2017, ["year", "month", "day"], "tmpc", "daily mean temp").drop(
163         columns=["year"])
164     df_daily_2017 = compute_mean(df_2017, ["month", "day"], "tmpc", "daily average temperature
165         (2017)")
166     #3.2 Mean temperatures over all years before 2017
167     df_mean = compute_mean(df_daily, ["month", "day"], "daily mean temp", "mean temp (before 2017)")
168
169     #4 Load consumption data & daily average
170     df_cons = load_consumption_data("Consumption deviation 2017.csv")
171     df_cons_daily = compute_mean(df_cons, ["month", "day"], "MW", "Avg. daily cons. deviation (2017)
172         ")
173
174     #5. Add mean temp, temp 2017 and temp deviation
175     df_cons_daily = df_cons_daily.merge(df_mean, on = ["month", "day"], how="left")
176     df_cons_daily = df_cons_daily.merge(df_daily_2017, on = ["month", "day"], how="left")
177     df_cons_daily["Temperature deviation (2017)"] = df_cons_daily["daily average temperature
178         (2017)"] - df_cons_daily["mean temp (before 2017)"]
179
180     #6. separate tables for summer and winter
181     summer = df_cons_daily[df_cons_daily["month"].isin([6, 7, 8])]
182     winter = df_cons_daily[df_cons_daily["month"].isin([12, 1, 2])]
183
184     #7. Calculate correlation
185     print("")
186     print("Winter: ", end="")
187     corr_winter = corr(winter, "Temperature deviation (2017)", "Avg. daily cons. deviation (2017)"
188         )
189     print("Summer: ", end="")
190     corr_summer = corr(summer, "Temperature deviation (2017)", "Avg. daily cons. deviation (2017)"
191         )
192
193     #8. Scatterplot, linear regression and plot
194     scatterplot(winter, summer, "Temperature deviation (2017)", "Avg. daily cons. deviation (2017)"
195         )
196     lin_reg_statsmodels(summer, "Temperature deviation (2017)", "Avg. daily cons. deviation (2017)"
197         , "Summer")
198     lin_reg_statsmodels(winter, "Temperature deviation (2017)", "Avg. daily cons. deviation (2017)"
199         , "Winter")
200
201     if __name__ == "__main__":
202         main()

```